DBMS TASK 7

BrightMartDB – SQL Reports

Prepared from: dbms task 7.sql

REPORT PACKAGE

No.	Deliverable	Contents
1	Product Search Report	Product details, price filters, category search and stock availability.
2	Customer Information Report	Customer list, city-wise count, customer orders and registration sorting.
3	Basic Business Analysis Report	Customer and order counts, order status summary, top-priced products and review counts.
4	Screenshot Checklist	Queries for which execution-result screenshots should be captured.

Important: The supplied SQL file contains SELECT queries, but no table-creation statements, INSERT records, or captured query results. Therefore, this PDF describes the report outputs and the SQL used; it does not invent product names, customer details, or numerical business results. Run the queries against your BrightMartDB database to populate the actual results.

1. Product Search Report

Objective: Retrieve product information from BrightMartDB to support product lookup, price filtering, category-based searching and stock checking.

Report fields

Field	Description
Product_ID	Unique identifier for each product (where selected by the query).
Product_Name	Name of the product.
Price	Listed product price.
Category_ID	Category identifier used for category filtering.
Stock_Quantity	Quantity currently recorded in inventory.
Stock_Status	Inventory status, when selected by the query.

SQL queries used

```
-- Product name and price
SELECT Product_Name, Price
FROM Product;

-- Products priced above 5000
SELECT * FROM Product
WHERE Price > 5000;

-- Products within the 1000–5000 price range
SELECT * FROM Product
WHERE Price BETWEEN 1000 AND 5000;

-- Products in category 1
SELECT * FROM Product
WHERE Category_ID = 1;

-- Search product names containing 'Mobile'
SELECT * FROM Product
WHERE Product_Name LIKE '%Mobile%';

-- Product and available stock
SELECT Product.Product_ID,
       Product.Product_Name,
       Inventory.Stock_Quantity
FROM Product
JOIN Inventory
  ON Product.Product_ID = Inventory.Product_ID
WHERE Inventory.Stock_Quantity > 0;
```

Expected report output

The executed queries should display matching product records. The stock query lists products with a positive inventory quantity. Actual rows and values depend on the records stored in BrightMartDB.

2. Customer Information Report

Objective: View customer records and summarize customer distribution by city. The SQL file also includes a query connecting customers with their orders.

Report fields

Field	Description
Customer_ID	Unique customer identifier.
Customer_Name	Customer name.
City	City recorded for the customer.
Registration_Date	Date the customer registered.
Order_ID / Order_Date	Order identifier and order date, when joined with Orders.
Total_Amount	Total value recorded for the order.

SQL queries used

```
-- All customer records
SELECT * FROM Customer;

-- Customers from Chennai
SELECT * FROM Customer
WHERE City = 'Chennai';

-- Customers sorted alphabetically
SELECT * FROM Customer
ORDER BY Customer_Name ASC;

-- Number of customers in each city
SELECT City, COUNT(*) AS Total_Customers
FROM Customer
GROUP BY City;

-- Customers with their order information
SELECT Customer.Customer_ID,
       Customer.Customer_Name,
       Orders.Order_ID,
       Orders.Order_Date,
       Orders.Total_Amount
FROM Customer
JOIN Orders
  ON Customer.Customer_ID = Orders.Customer_ID;

-- Most recently registered customers first
SELECT *
FROM Customer
ORDER BY Registration_Date DESC;
```

Expected report output

The report should show customer details, city-wise customer counts and customer order information. The SQL file does not provide actual customer records, so names and counts must be taken from the database query results.

3. Basic Business Analysis Report

Objective: Use aggregate and sorting queries to provide a basic overview of customers, orders, product pricing and product reviews.

Key business indicators

Indicator	SQL calculation / meaning
Total Customers	COUNT(*) from Customer.
Customers by City	COUNT(*) grouped by City.
Total Orders	COUNT(*) from Orders.
Completed / Delivered Orders	Count where Order_Status is Completed or Delivered.
Pending Orders	Count where Order_Status is Pending.
Cancelled Orders	Count where Order_Status is Cancelled.
Top 5 Highest-priced Products	Products sorted by Price descending, limited to five rows.
Product Review Counts	Review count per product, sorted from highest to lowest.

SQL queries used

```
-- Total customers
SELECT COUNT(*) AS Total_Customers
FROM Customer;

-- Customers grouped by city
SELECT City, COUNT(*) AS Total_Customers
FROM Customer
GROUP BY City;

-- Total orders
SELECT COUNT(*) AS Total_Orders
FROM Orders;

-- Completed and delivered orders
SELECT COUNT(*) AS Completed_Orders
FROM Orders
WHERE Order_Status IN ('Completed', 'Delivered');

-- Pending orders
SELECT COUNT(*) AS Pending_Orders
FROM Orders
WHERE Order_Status = 'Pending';

-- Cancelled orders
SELECT COUNT(*) AS Cancelled_Orders
FROM Orders
WHERE Order_Status = 'Cancelled';

-- Five highest-priced products
SELECT *
FROM Product
ORDER BY Price DESC
LIMIT 5;

-- Number of reviews for each product
SELECT Product.Product_Name,
       COUNT(Review.Review_ID) AS Total_Reviews
FROM Product
LEFT JOIN Review
  ON Product.Product_ID = Review.Product_ID
GROUP BY Product.Product_ID, Product.Product_Name
```

```
ORDER BY Total_Reviews DESC;
```

Analysis notes

Use the returned counts to understand customer volume and order status distribution. The top-five product query identifies the highest-priced products, while the review-count query indicates which products have received the most reviews. No numeric conclusions can be stated until these queries are executed on the actual database.

4. Query Output Screenshot Checklist

Capture screenshots in your SQL application after executing each query. Each screenshot should show the SQL statement and its result grid, with column headings visible. If a query returns no rows, capture the empty result as evidence rather than changing the query.

No.	Query / result to capture	Done
1	All customers and all products.	■
2	Product name and price; products above 5000.	■
3	Available inventory; Chennai customers; completed/delivered orders.	■
4	Average product rating greater than 4.	■
5	Customer and product sorting; distinct categories, payment modes and cities.	■
6	Price range, category search, product-inventory join and Mobile search.	■
7	Customer-order join; product-category join; customer/product order details.	■
8	Combined filters, date range, category IN query and inventory status.	■
9	Customer/order counts and order-status counts.	■
10	Top 5 products, review counts and products with available stock.	■

Submission checklist

■ Original file: dbms task 7.sql
■ Query execution screenshots
■ Product Search Report (this PDF)
■ Customer Information Report (this PDF)
■ Basic Business Analysis Report (this PDF)

Source note: All SQL examples and report topics in this document are based on the uploaded dbms task 7.sql file. This PDF is a report-format document; it is not a database execution log.